

四電一丙 數位邏輯實習

第六次實習作業報告

組員 1 學號:B812111001 姓名: 張博翔

組員 2 學號: 114310326 黃子軒

組員 1 於此作業中的貢獻:解題、digital、Quartus、協助隊友排錯、拍照(作業一)、作業報告(二次編排、修改與美編)

組員 2 於此作業中的貢獻:解題、digital、quartus、拍照(作業二)、作業報告(初步編排、複審內容)

I. 實驗目的

A. 作業 1

請利用以下兩種方式實作 S-R 正反器，方法一：利用 NOR 邏輯閘實作投影片第 3 頁的 S-R Latch，將完成的 S-R Latch 模組化之後，實作投影片第 10 頁的 S-R 正反器。方法二：利用 VHDL Process 實作 S-R 正反器，比較投影片第 10 頁的波型圖與兩個方法實作 S-R 正反器的波形圖，請嘗試修正投影片第 10 頁中紅色區域的錯誤。(加分項目 +10)

B. 作業 2

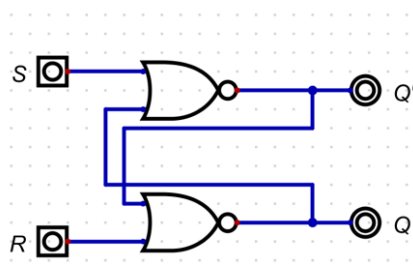
作業題 2：請利用以下兩種方式實作 A-B 正反器，方法一利用 VHDL Process 實作 A-B 正反器，方法二利用作業題 1 的 S-R 正反器實作 A-B 正反器。

II. 設計程序

A. 作業 1

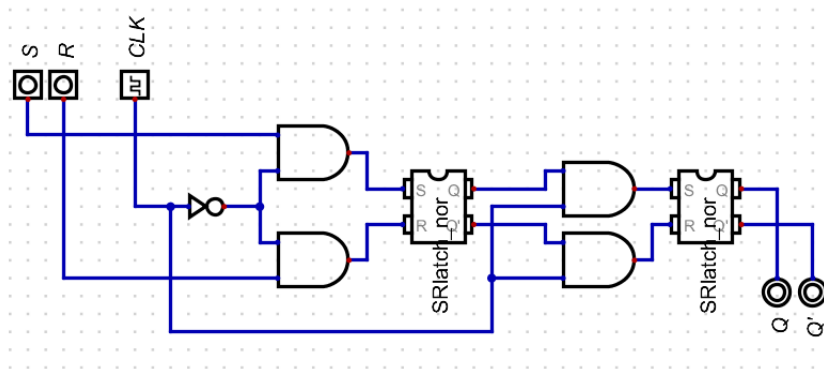
方法一

利用 NOR 邏輯閘實作 S-R Latch 並模組化，運用兩個 Latch 構建 Flip-Flop。



圖一: S-R Latch NOR 閘電路

相連完成兩個 Latch 與邏輯閘連結，組合成具備 CLK 的 S-R 正反器，CLK = 1 時 AND 閘打開 SR 可以進入 Master 更新，反相後的 CLK，當 CLK = 0，AND 閘打開接收第一級輸出 Slave 更新。



圖二: S-R 正反器

方法二

利用 VHDL 的程序來進行，使用 Process 寫出利用 process(CLK) 來監測時脈訊號的變化，加上今天所學新的語法，做出 S-R 正反器，並比較投影片第 10 頁的波型圖與兩個方法實作 S-R 正反器的波形圖。

表一: S-R 正反器 VHDL

```

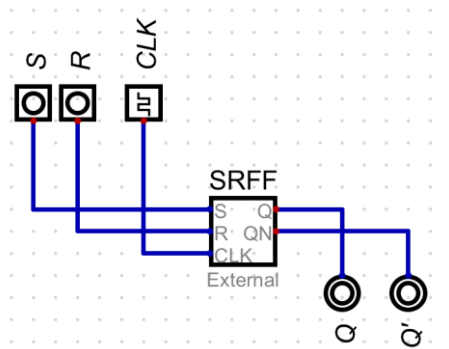
1. LIBRARY ieee;
2. USE ieee.std_logic_1164.all;
3. USE ieee.numeric_std.all;
4.
5. entity SRFF is
6. port (S, R, CLK: in std_logic;
7.       Q, QN: out std_logic);
8. end SRFF;
9.
10. architecture SRFF1 of SRFF is
11. signal Qint: std_logic:= '0';
12.
13. begin
14.   Q <= Qint;          -- output Q and QN to port
15.   QN <= not Qint;
16.   process (CLK)
17.   begin
18.     if CLK'event and CLK = '1' then
19.       if S='0' and R='0' then Qint <= Qint;
20.       elsif S='1' and R='0' then Qint <= '1';
21.       elsif S='0' and R='1' then Qint <= '0';
22.       end if;
23.     end if;
24.   end process;
25. end SRFF1;

```

1~3 行是 Library 宣告區，載入資料庫，定義邏輯閘功能與語法。

5~8 行是介面定義，定義有輸入訊號 S 與 R、CLK、輸出訊號 Q、Q'。

10~25 行是主要程式，定義 CLK 是 1 的時候如果 S 是 R 是什麼，那輸出 Q 要是什麼，否則往後下一個條件。



圖三: S-R 正反器 VHDL 模擬電路

B. 作業 2

方法一

這次作業二是利用 VHDL Process 寫法，實作 AB 正反器，依照老師投影片提供的真值表，加上今天所學新的語法，利用 Process (CLK)來監測時脈訊號的邊緣觸發，然後因為會需要判別所以會用到使 if...elsif...else 條件式。

表二: AB 正反器真值表

0	0	0	歸零
0	1	Q	保持原狀態
1	0	Q'	相反狀態
1	1	1	設為 1

依據投影片提供的範例編寫 VHDL Process，並進行編譯

表三: AB 正反器程式碼

```
1. LIBRARY ieee;
2. USE ieee.std_logic_1164.all;
3. USE ieee.numeric_std.all;
4.
5. entity AB_FlipFlop is port (
6.   A, B : in std_logic;
7.   CLK : in std_logic;
8.   Q : out std_logic
9. );
10. );
11. end AB_FlipFlop;
12.
13. architecture Behavioral of AB_FlipFlop is
14.   signal Q_int : std_logic := '0';
15.   begin
16.     Q <= Q_int;
17.
18.     process (CLK)
19.     begin
20.       if rising_edge(CLK) then
21.         if A = '0' and B = '0' then
22.           Q_int <= '0';
23.         elsif A = '0' and B = '1' then
24.           Q_int <= Q_int;
25.         elsif A = '1' and B = '0' then
26.           Q_int <= not Q_int;
27.         elsif A = '1' and B = '1' then
28.           Q_int <= '1';
29.         end if;
30.       end if;
31.     end process;
32.   end Behavioral;
```

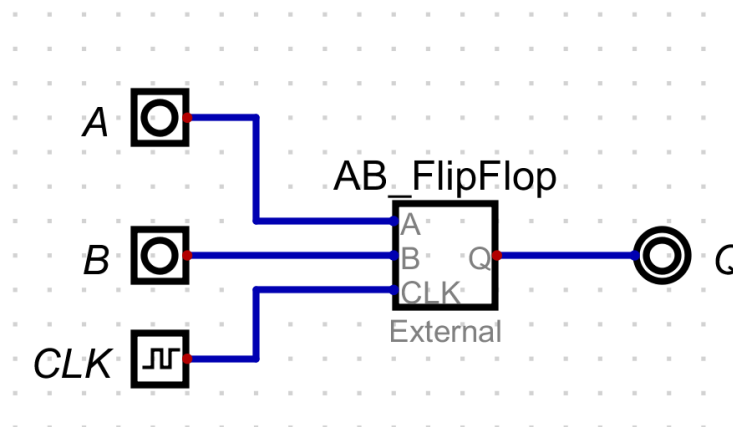
VHDL 程式碼講解：

1~3 行是 Library 宣告區，載入資料庫，定義邏輯閘功能與語法。

5~11 行是介面定義，定義有輸入訊號 A 與 B、CLK、輸出訊號 Q。

13~16 行是宣告內部訊號來儲存當前狀態，因為輸出腳位 Q 不能一下是輸出一下是出入，需要設定一個內部訊號。

19~34 行是程式的核心運算區，定義輸入與輸出間關係，如圖先進行偵測時脈的上升邊緣觸，如果 A=0,B=0 輸出 Q_int=0， elsif A = 0 , B = 1 輸出 Q_int = Q_int， elsif A = 1 , B = 0 Q_int= Q' A = 1, B = 1， Q_int= 1 輸出為 1 模擬實際電路中邏輯閘關係。



圖四: AB 正反器模擬電路

方法二

第二種方法是利用作業一所實作的 S-R 正反器來建構 A-B 正反器，透過元件接線的方式達成設計目標，首先需要建立 S 和 R 的邏輯方程式， $S = A \cdot B + A \cdot B' \cdot Q'$ 、 $R = A' \cdot B' + A \cdot B' \cdot Q$ ，把邏輯方程式接線完後，再接上 SRFF 正反器，接完後測試其功能正確性。

表四: AB 正反器真值表

A	B	Q
0	0	Q = 0
0	1	保持原態
1	0	Q 反轉
1	1	Q = 1

表五: SR 正反器真值表

S	R	Q
0	0	保持原態
0	1	Q = 0
1	0	Q = 1
1	1	矛盾

表六:SRFF 正反器程式碼

```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity SRFF is
5.     Port (
6.         S : in STD_LOGIC;
7.         R : in STD_LOGIC;
8.         CLK : in STD_LOGIC;
9.         Q : out STD_LOGIC;
10.        QN : out STD_LOGIC
11.    );
12. end SRFF;
13.
14. architecture Behavioral of SRFF is
15.     signal Q_int : STD_LOGIC := '0';
16. begin
17.
18.     process(CLK)
19.     begin

```

```

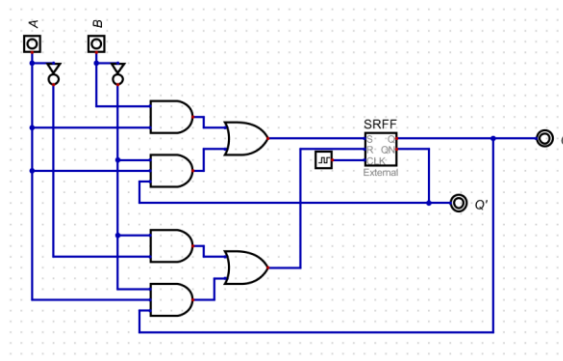
20.  if rising_edge(CLK) then
21.      if (S = '1' and R = '0') then
22.          Q_int <= '1';
23.      elsif (S = '0' and R = '1') then
24.          Q_int <= '0';
25.      elsif (S = '0' and R = '0') then
26.          Q_int <= Q_int; -- Hold
27.      else
28.          Q_int <= 'X'; -- invalid (S=1,R=1)
29.      end if;
30.  end if;
31. end process;
32.
33. Q <= Q_int;
34. QN <= not Q_int;
35.
36. end Behavioral;

```

1~2 行是 Library 宣告區，載入資料庫，定義邏輯閘功能與語法。

4~12 行是介面定義，定義有輸入訊號 S 與 R、CLK、輸出訊號 Q、Q'

14~36 行是程式的核心運算區，定義輸入與輸出間關係，如圖先進行偵測時脈的上升邊緣觸，如果 S=1,B=0 輸出 =1，elsif A = 0, B = 1 輸出 Q_int = 0，elsif S = 0, R = 0 Q_int= Q' A = 1, B = 1, Q_int= X，模擬實際電路中邏輯閘關係。



圖五: AB--SRFF 正反器模擬電路

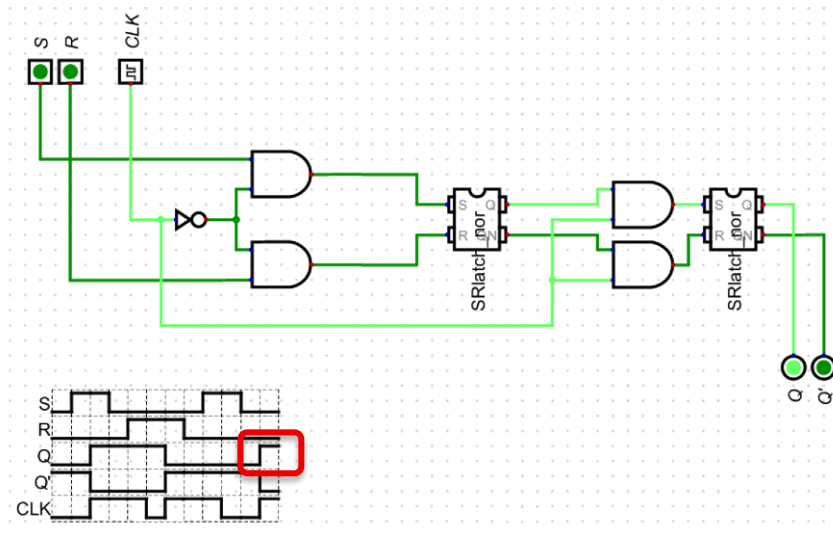
III. 模擬驗證

A. 作業 I

模擬測試，測試並比較與投影片第 10 頁的波型圖與兩個方法實作 S-R 正反器的波形圖，以及嘗試修正投影片第 10 頁中紅色區域的錯誤。

方法一

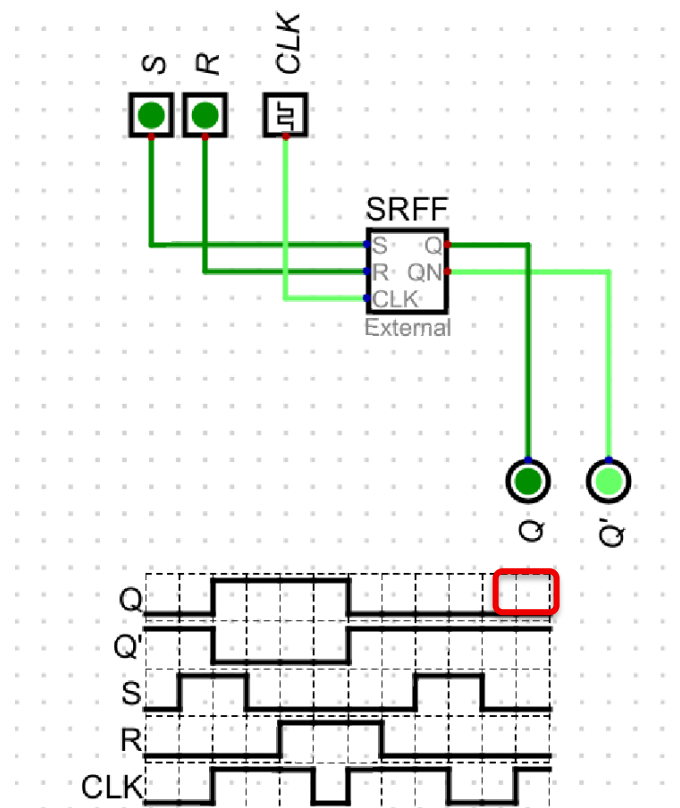
分析波形圖，當 SR 為 01，CLK 為 1 輸出為 0，SR 為 10，CLK 為 1 輸出為 1，SR 為 00，CLK 為 1 輸出一樣是 1，SR 為 01，CLK 為 1 輸出變成 0，最後 SR 為 00，CLK 為 1 輸出維持 0。



圖六: SR 正反器模擬電路

方法二

分析波形圖，時序和輸入都和方法一樣，方法一與方法二產生的 Q 輸出波形卻有不同。這是因為 NOR 閘接線在負緣處變化會被讀入，運用行為描述可以以時脈正緣為變化條件改善問題。

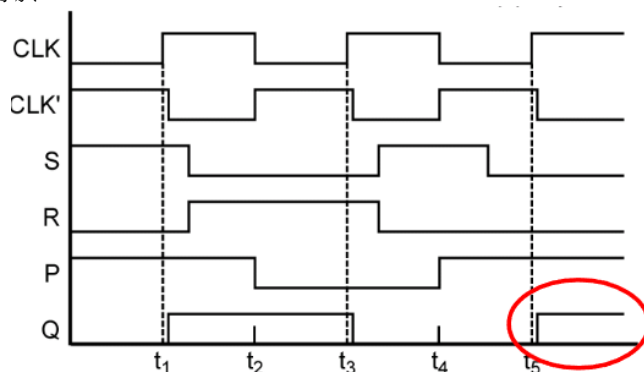


圖七: SR 正反器模擬電路

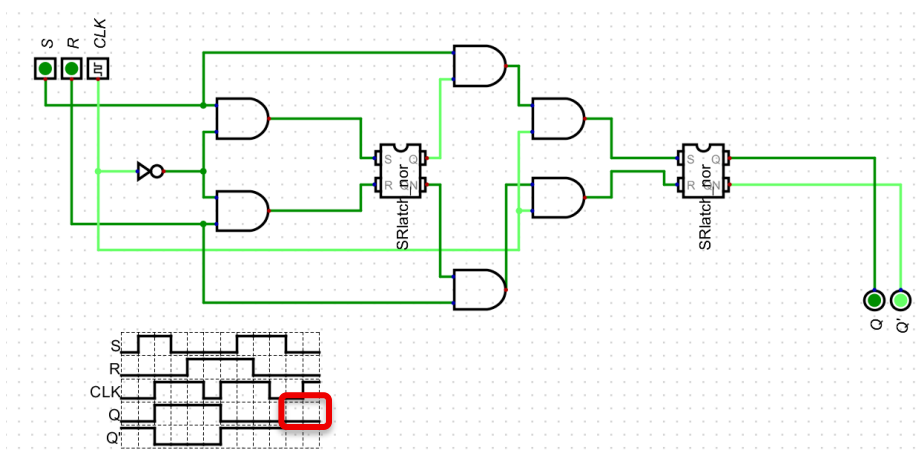
加分題

在投影片第 10 頁的波型圖中從 t3 到 t5 之間輸出都是在 0 的位子，那輸出 Q 在 t5 時也要是 0，但是邏輯錯誤從 0 跳變成了 1，投影片第 10 頁的波型圖中紅圈處在 S=R=0 時讓 Q 發生跳變，但正確的 S-R flip-flop 在此條件下應維持

原值。原因是因為 $S=R=0$ 為「保持狀態」，且 master-slave 結構在非觸發時不會將新值傳到輸出，錯誤忽略了時脈相位造成的鎖存隔離效果，導致誤以為在 $S=R=0$ 時 Q 仍可能更新，以下是修正後，和投影片第 10 頁的波型圖其相同測試方式，最後 t_5 輸出 Q 等於 0。



圖八: 投影片第 10 頁的波型圖

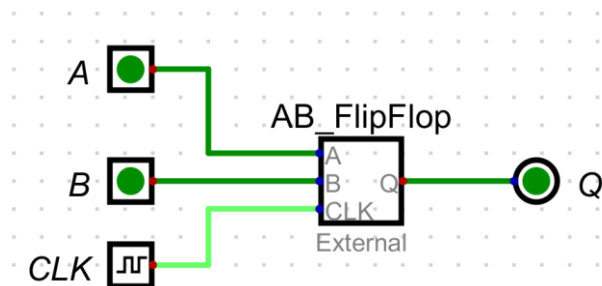


圖九: 修正過後的 SRFF

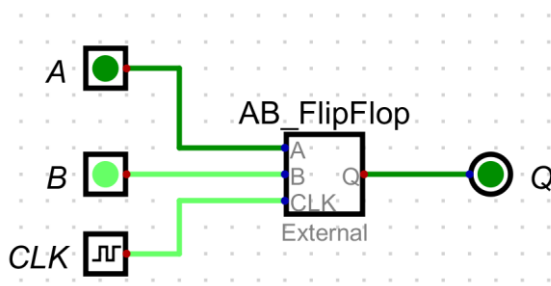
B. 作業 2

方法一

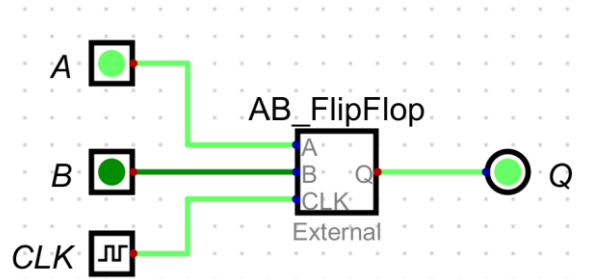
AB 正反器：Input A、B、CLK；Output Q，當 $A=0$ $B=0$ 在 $CLK=1$ 時， $Q=0$ 、當 $A=0$ $B=1$ 在 $CLK=1$ 時， $Q=Q$ (原態)，當 $A=1$ $B=0$ 在 $CLK=1$ 時， $Q=Q'$ (反態)，當 $A=1$ $B=1$ 在 $CLK=1$ 時， $Q=1$ ，下圖為模擬圖，結果都正確。



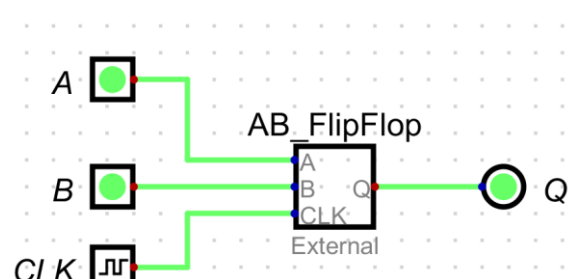
圖十. Input: $A=0$ 、 $B=0$ $CLK=1$ / Output: $Q=0$
因為是 00 所以輸出是 0



圖十一. Input: $A=0$ 、 $B=1$ $CLK=1$ / Output: $Q=Q$
因為上一部驟是 0 所以原態是 0



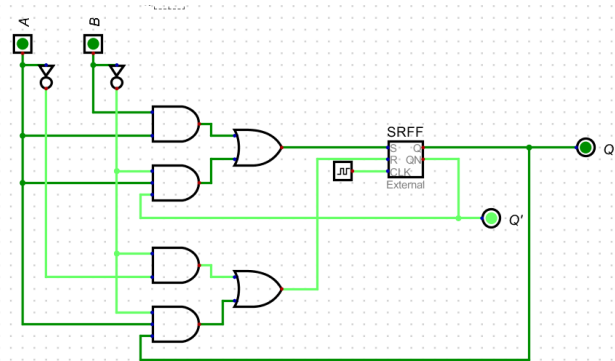
圖十二. Input:A=1、B=0 CLK=1 / Output:Q=1
因為上一部驟是 0 所以反態是 1



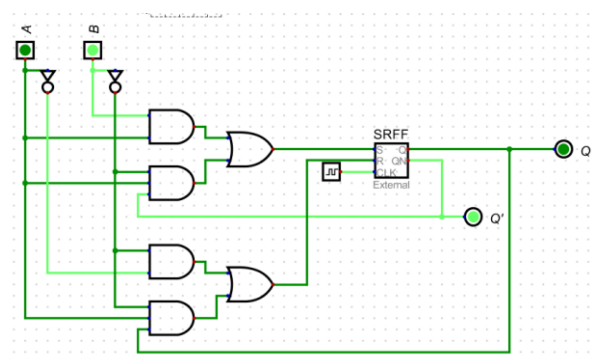
圖十三. Input:A=1、B=1 CLK=1 / Output:Q=1
因為是 11 所以輸出是 1

作業2 方法二

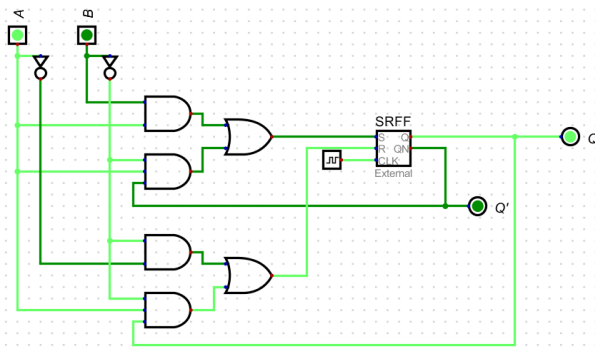
S-R 正反器來建構 A-B 正反器，一樣的測試數據 Input A、B、CLK；Output Q，當 A = 0 B = 0 在 CLK = 1 時，Q = 0、當 A = 0 B = 1 在 CLK=1 時，Q=Q(原態)，當 A = 1 B = 0 在 CLK=1 時，Q=Q'(反態)，當 A = 1 B = 1 在 CLK=1 時，Q = 1，不同的是有新增邏輯方程式， $S = A \cdot B + A \cdot B' \cdot Q'$ 、 $R = A' \cdot B' + A \cdot B' \cdot Q$ ，下圖為模擬圖，結果和方法一都正確。



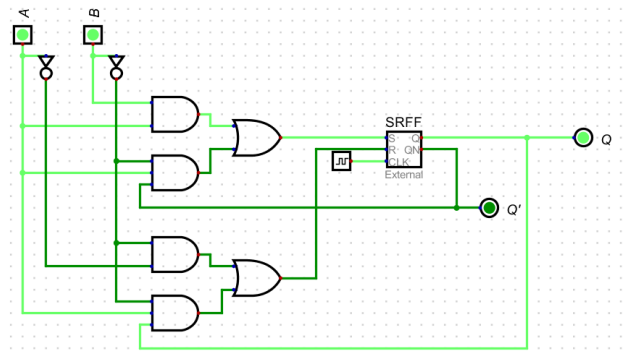
圖十四. Input:A=0、B=0 CLK=1 / Output:Q=0



圖十五. Input:A=0、B=1 CLK=1 / Output:Q=Q
因為上一步驟是 0 所以原態是 0



圖十六. Input:A=1、B=0 CLK=1 / Output:Q=1
因為上一步驟是 0 所以反態是 1

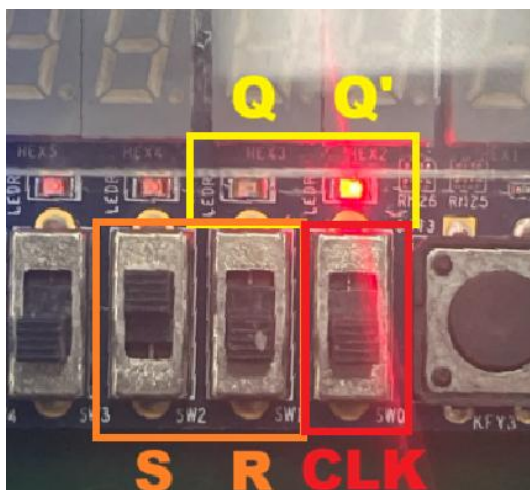


圖十七. Input:A=1、B=1 CLK=1 / Output:Q=1

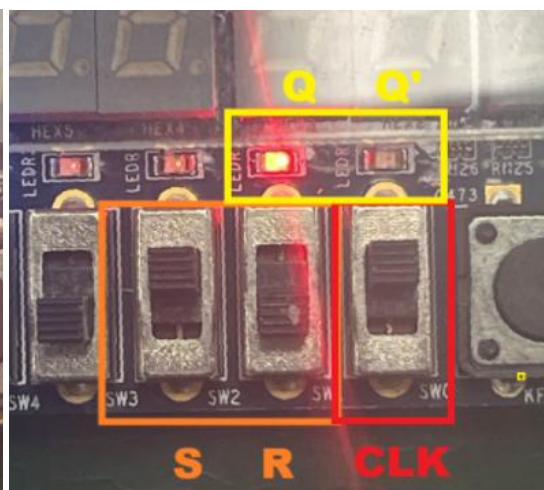
IV.實驗結果與成果討論

A. 作業1

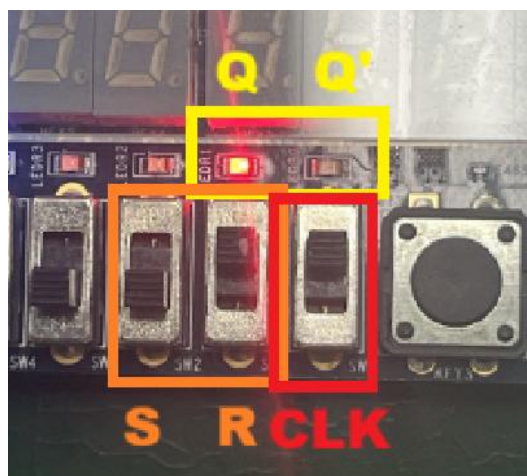
SR 正反器：Input S、R、CLK；Output Q、Q'；Input S、R、CLK 依序為 SW2、SW1、SW0，Output Q、Q'為 LED 1、LED 0，以下為實驗結果。



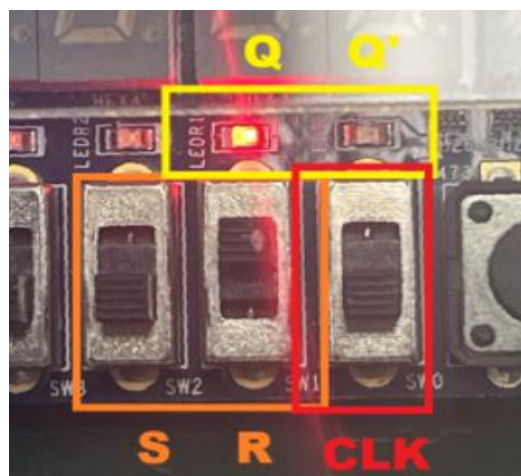
圖十八. Input:SR=10 CLK=0 / Output:=Q'



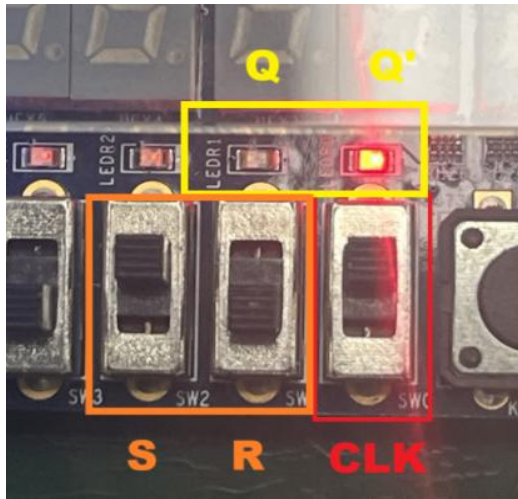
圖十九. Input:SR=10 CLK=1 / Output:=Q



圖二十. Input:SR=01 CLK=1 / Output:=Q



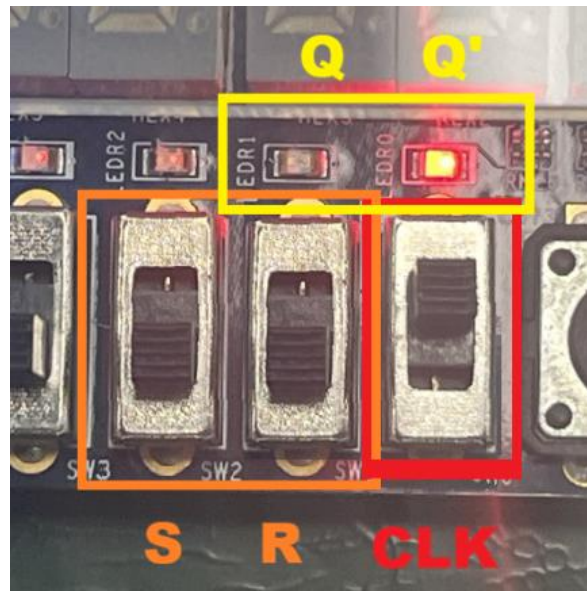
圖二十一. Input:SR=01 CLK=0 / Output:=Q



圖二十二. Input:SR=10 CLK=1 / Output:=Q'



圖二十三. Input:SR=00 CLK=0 / Output:=Q'



圖二十四. Input:SR=00 CLK=1 / Output:=Q'

B. 作業 2

AB 正反器：Input A、B、CLK；Output Q；Input A、B、CLK 為 SW2、SW1、SW0，Output Q 為 LED0，以下為實驗結果。

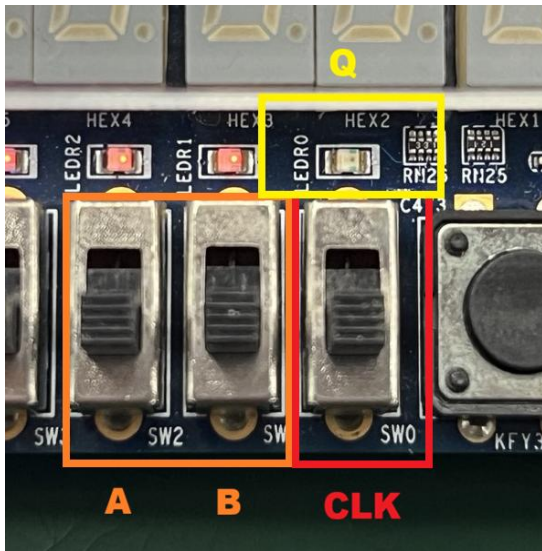


圖. Input:A=0 、 B=0 CLK=0 / Output:Q=0

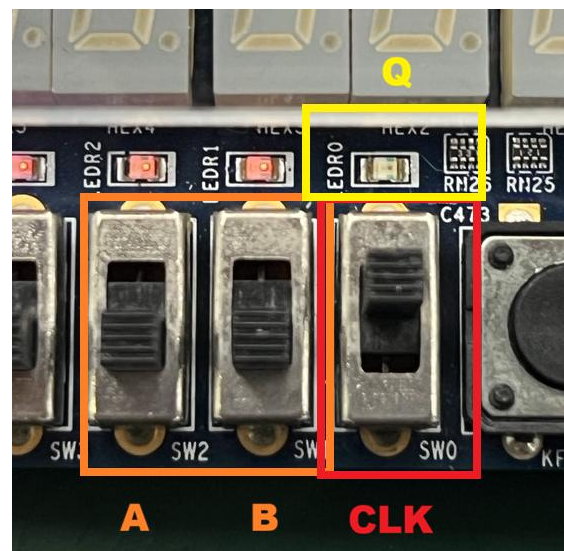


圖. Input:A=0 、 B=0 CLK=1 / Output:Q=0

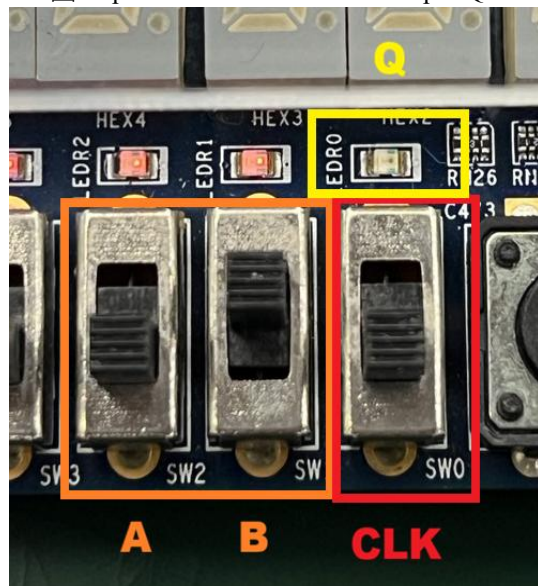


圖. Input:A=0 、 B=1 CLK=0 / Output:Q=0

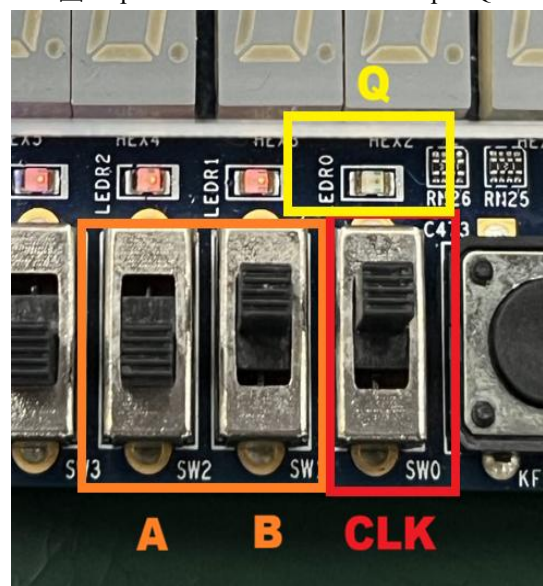


圖. Input:A=0 、 B=1 CLK=1 / Output:Q=0

這邊為零是因為上一個 Q 為零保持原態

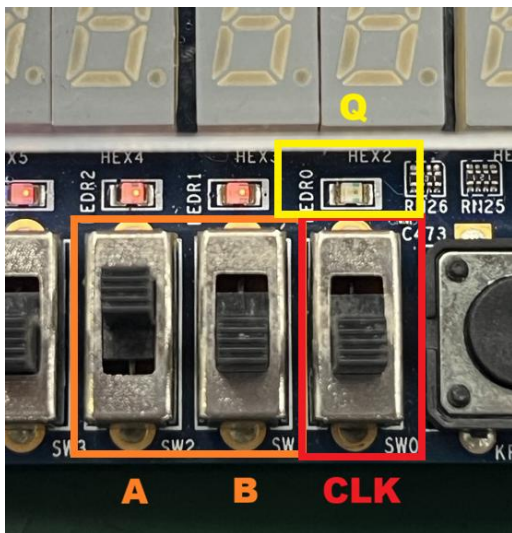


圖. Input:A=1、B=0 CLK=0 / Output:Q=0

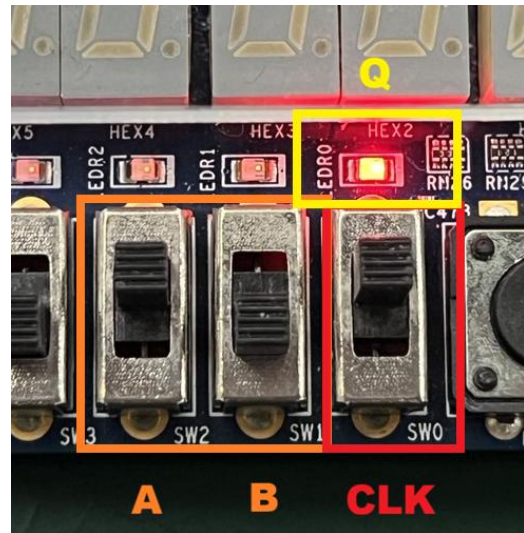


圖. Input:A=1、B=0 CLK=1 / Output:Q=1

這邊為一是因為上一個 Q 為零，反態變一

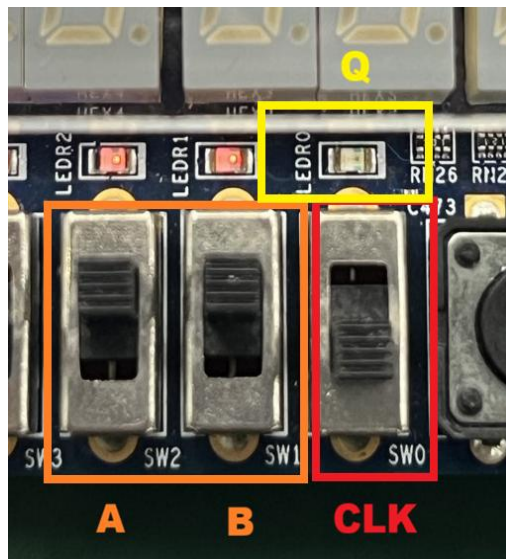


圖. Input:A=1、B=1 CLK=0 / Output:Q=0

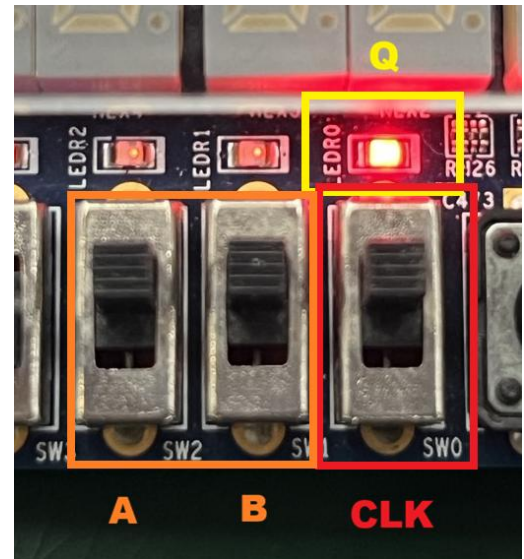


圖. Input:A=1、B=1 CLK=1 / Output:Q=1

V. 實驗心得

組員 1：這次作業讓我更清楚 S-R Flip-Flop，透過加分題，也加深了我對 master-slave 時序控制的理解。

組員 2：這次作業一開始覺得有點難，尤其是時序的部分不太好理解。不過在實際畫波形後，才比較清楚。

參考文獻

- [1] DLE 07 數位邏輯實習 VHDL(I)